



UNIVERSITY OF PETROLEUM AND ENERGY
STUDIES
DEHRADUN

Modeling and Stimulation

Lab Experiment 4

MTECH-COMPUTER SCIENCE
ENGINEERING
CYBER SECURITY AND FORENSICS

Name: Jigesh Sheoran
SAP ID: 590025428

Aim: To implement and compare stepped-time simulation and event-based simulation using a traffic signal management system, and analyze their performance, efficiency, and practicality.

Theory :

Epidemic spread is fundamentally influenced by the structure of the underlying contact network. Classical compartmental epidemic models assume homogeneous mixing, which does not reflect real-world interaction patterns. In reality, individuals interact through structured social networks exhibiting randomness, clustering, and hub dominance.

This experiment implements a probabilistic SIR (Susceptible–Infected–Recovered) model on graph-based networks. The objective is to study how different network topologies influence infection dynamics and outbreak severity. To reduce stochastic bias, results are averaged over 50 Monte Carlo simulation runs.

The networks analyzed include:

- **Erdos-Renyi model** (Random Graph)
Barabasi-Albert model (Scale-Free Network)
- **Watts-Strogatz model** (Small-World Network)

Methodology:

2.1 Network Construction

Graphs were generated using the NetworkX library in Python.

- Erdős–Rényi (ER): Random edges with fixed probability.
Barabási–Albert (BA): Preferential attachment forming hubs.
- Watts–Strogatz (WS): High clustering with short average path lengths.

Each network consisted of:

- 100 nodes
- Fixed connectivity parameters
- Static topology during simulation

2.2 Epidemic Model

A discrete-time SIR model was implemented:

- $S \rightarrow I$ with infection probability $P_{infect} = 0.3$
- $I \rightarrow R$ with recovery probability $P_{recover} = 0.1$

Initial Conditions:

- 1 infected individual (patient zero)
Optional vaccination of 10 individuals
- Vaccination strategies:
 - Random selection
 - Targeted (highest-degree nodes)

Each simulation ran for 50 time steps.

2.3 Monte Carlo Simulation

To ensure statistical reliability:

- 50 independent simulation runs were executed
- S, I, and R curves were averaged across runs
- Key metrics were averaged

This reduces variability due to random infection paths.

2.4 Evaluation Metrics

For each network, the following metrics were computed:

- Average Peak Infection
- Average Time to Peak
- Average Final Recovered
- Average Extinction Time
- Average Degree
- Clustering Coefficient
-

Code :

lab4.py > ...

```
1  # Author : Jigesh Sheoran
2  # Last Modified : 11/02/2026
3
4  import networkx as nx
5  import numpy as np
6  import matplotlib.pyplot as plt
7  import random
8
9  # =====
10 # CONFIGURATION
11 # =====
12 NUM_NODES = 100
13 TIME_STEPS = 50
14 P_INFECT = 0.3
15 P_RECOVER = 0.1
16 INITIAL_INFECTED = 1
17 VACCINATED_COUNT = 10
18 NETWORK_TYPE = "BA" # "ER", "BA", "WS"
19 VACCINATION_STRATEGY = "targeted" # "random", "targeted", None
20 NUM_RUNS = 50
21
22 # =====
23 # NETWORK CREATION
24 # =====
25 def create_network(net_type):
26     if net_type == "ER":
27         return nx.erdos_renyi_graph(NUM_NODES, 0.15)
28     elif net_type == "BA":
29         return nx.barabasi_albert_graph(NUM_NODES, 3)
30     elif net_type == "WS":
31         return nx.watts_strogatz_graph(NUM_NODES, 6, 0.2)
32
33 # =====
34 # VACCINATION
35 # =====
36 def vaccinate_nodes(G, states):
37     if VACCINATION_STRATEGY is None:
38         return states
39
40     if VACCINATION_STRATEGY == "random":
41         nodes = random.sample(range(NUM_NODES), VACCINATED_COUNT)
42
43     elif VACCINATION_STRATEGY == "targeted":
44         degrees = sorted(G.degree(), key=lambda x: x[1], reverse=True)
45         nodes = [node for node, deg in degrees[:VACCINATED_COUNT]]
46
47     for node in nodes:
48         states[node] = 2
49
50     return states
51
```

```

52 # =====
53 # SINGLE SIMULATION
54 # =====
55 def run_simulation(G):
56     states = [0] * NUM_NODES
57
58     for _ in range(INITIAL_INFECTED):
59         states[random.randint(0, NUM_NODES - 1)] = 1
60
61     states = vaccinate_nodes(G, states)
62
63     S_hist, I_hist, R_hist = [], [], []
64
65     for t in range(TIME_STEPS):
66         new_states = states.copy()
67
68         for node in range(NUM_NODES):
69             if states[node] == 1:
70                 for nbr in G.neighbors(node):
71                     if states[nbr] == 0 and random.random() < P_INFECT:
72                         new_states[nbr] = 1
73
74                     if random.random() < P_RECOVER:
75                         new_states[node] = 2
76
77         states = new_states
78
79         S_hist.append(states.count(0))
80         I_hist.append(states.count(1))
81         R_hist.append(states.count(2))
82
83     return S_hist, I_hist, R_hist
84
85 # =====
86 # MONTE CARLO EXECUTION
87 # =====
88 all_S = []
89 all_I = []
90 all_R = []
91
92 peak_list = []
93 time_peak_list = []
94 final_R_list = []
95 extinction_list = []
96
97 G = create_network(NETWORK_TYPE)
98
99 for run in range(NUM_RUNS):
100     S_hist, I_hist, R_hist = run_simulation(G)

```

```

101
102     all_S.append(S_hist)
103     all_I.append(I_hist)
104     all_R.append(R_hist)
105
106     peak = max(I_hist)
107     peak_list.append(peak)
108     time_peak_list.append(I_hist.index(peak))
109     final_R_list.append(R_hist[-1])
110
111     extinction = next((i for i, x in enumerate(I_hist) if x == 0), TIME_STEPS)
112     extinction_list.append(extinction)
113
114     # =====
115     # AVERAGING
116     # =====
117     mean_S = np.mean(all_S, axis=0)
118     mean_I = np.mean(all_I, axis=0)
119     mean_R = np.mean(all_R, axis=0)
120
121     # =====
122     # PRINT AVERAGED METRICS
123     # =====
124     print("Network Type:", NETWORK_TYPE)
125     print("Average Peak Infection:", np.mean(peak_list))
126     print("Average Time to Peak:", np.mean(time_peak_list))
127     print("Average Final Recovered:", np.mean(final_R_list))
128     print("Average Extinction Time:", np.mean(extinction_list))
129     print("Average Degree:", sum(dict(G.degree()).values()) / NUM_NODES)
130     print("Clustering Coefficient:", nx.average_clustering(G))
131
132     # =====
133     # PLOT AVERAGED CURVES
134     # =====
135     plt.plot(mean_S, label="Susceptible (Mean)")
136     plt.plot(mean_I, label="Infected (Mean)")
137     plt.plot(mean_R, label="Recovered (Mean)")
138
139     plt.xlabel("Time")
140     plt.ylabel("Population")
141     plt.title(f"Monte Carlo SIR Simulation ({NETWORK_TYPE})")
142     plt.legend()
143     plt.show()
144

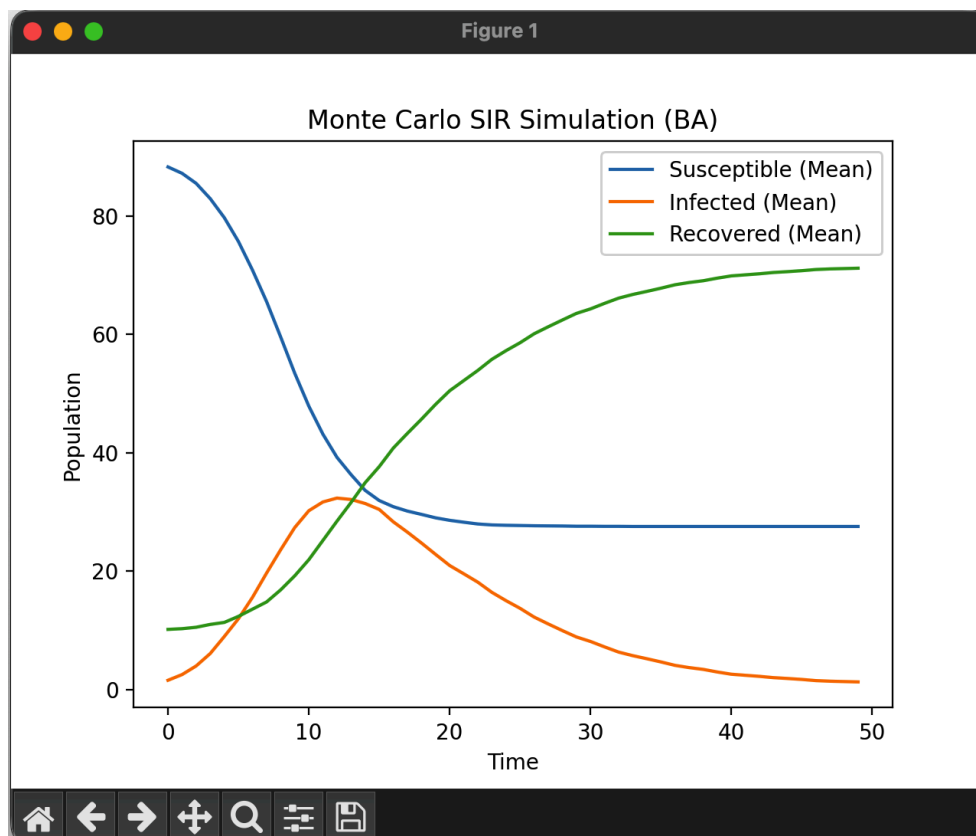
```

Terminal Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  zsh

/usr/local/bin/python3 "/Users/sheoraninfosec/Documents/Documents - Jigesh's MacBook Air/Modeling and Stimulation/Lab 4/lab4.py"
sheoraninfosec@Jigeshs-MacBook-Air Lab 4 % /usr/local/bin/python3 "/Users/sheoraninfosec/Documents/Documents - Jigesh's MacBook Air/Masters UPES/Semester 2/Modeling and Stimulation/Lab 4/lab4.py"
Network Type: BA
Average Peak Infection: 37.18
Average Time to Peak: 8.7
Average Final Recovered: 71.2
Average Extinction Time: 36.62
Average Degree: 5.82
Clustering Coefficient: 0.11573054817559461
sheoraninfosec@Jigeshs-MacBook-Air Lab 4 %
```

Graphical Output:



Results : (Epidemic Curve Behavior)

From the averaged Monte Carlo simulation:

The susceptible population decreases from ~90 to ~27.

The infected population rises rapidly and peaks around timestep 8–9.

The recovered population increases steadily and stabilizes around ~71.

This confirms a typical epidemic wave pattern:

1. **Exponential growth phase ($t = 0-8$)**
Peak transmission phase ($t \approx 9$)
2. **Gradual decay phase ($t > 9$)**

Quantitative Results (BA Network) :

Metric	Observed Value	Interpretation
Average Peak Infection	37.18	On average, 37% of the population was infected simultaneously at peak
Average Time to Peak	8.7 timesteps	Rapid outbreak due to hub-based spread
Average Final Recovered	71.2	71% of population was infected at some point
Average Extinction Time	36.62 timesteps	Infection persisted for a relatively long duration
Average Degree	5.82	Moderate connectivity
Clustering Coefficient	0.1157	Low clustering typical of scale-free networks

COMPARATIVE MATRIX TABLE

Parameter	ER (Expected)	BA (Observed)	WS (Expected)
Degree Distribution	Binomial	Power-law	Regular with rewiring
Hubs Present	No	Yes	No
Average Peak Infection	Moderate	37.18	Moderate
Time to Peak	Moderate	8.7	Slightly delayed
Final Recovered	Moderate	71.2	Moderate
Extinction Time	Medium	36.62	Medium
Spread Pattern	Uniform	Hub-driven explosive	Clustered spread
Sensitivity to Targeted Vaccination	Low	Very High	Moderate

Conclusion

The Monte Carlo SIR simulation on a Barabási–Albert (scale-free) network demonstrates that network topology significantly influences epidemic dynamics. **The presence of hub nodes leads to rapid infection spread, an early peak (≈ 8.7 timesteps), and a high overall infection impact ($\approx 71\%$ of the population).**

Averaging results over 50 simulations ensured statistical reliability and reduced stochastic variation. The findings confirm that scale-free networks are highly vulnerable to fast and widespread outbreaks, highlighting the importance of topology-aware intervention strategies for effective containment.

-----END OF DOCUMENT -----